appendix_react/01/app/javascript/contexts/schedule_context.ts

```
let scheduleChannel: Subscription
let favoritesChannel: Subscription

export const initialState = {
  scheduleDays: {},
  favorites: [],
  userId: null,
  favoritesChannel: null,
  textFilter: "",
  searchResults: [],
  favoritesVisible: true,
  mostRecentFavoriteId: null,
  removedFavoriteIds: [],
}
```

That's three elements that come from the server in the initial fetch (scheduleDays, favorites, and userId), two ActionCable channels (which can't be in the state object because they are not serializable), and five elements that we'll be using to store client-side state as we move along. The scheduleDays object also keeps local track of the state of the calendar filters, which isn't shared with the server.

We have to initialize the two ActionCable channels using calls that are very similar to the way we initialized the channel on the concert page. Then we dispatch a command to the reducer to fetch data. This is an asynchronous command, so we need to do it with a Redux thunk that grabs the data then dispatches to the actual reducer:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
export const fetchData = (): ScheduleThunk => {
  return async (dispatch) => {
    const response = await fetch("/schedule.json")
    const data = await response.json()
    dispatch({ type: "initFromData", data: data as IncomingScheduleData })
  }
}
```

All that does is make a call to the Rails app, convert the resulting JSON, and dispatch it to the actual reducer, which inserts the data into the state: